SkillSync — MongoDB Data Model

Modified Section 6: Database Collections (Adapted from PostgreSQL SRS)

Version 2.0 | MongoDB Edition

Overview of Changes

The original SRS specified a PostgreSQL relational schema with 12 tables and multiple junction tables. Since MongoDB is a document database, the schema has been redesigned with these core principles:

1. Eliminate junction tables by embedding arrays directly inside parent documents.
2. Replace UUID Primary Keys with MongoDB ObjectId (_id) on all collections.
3. Remove Foreign Key constraints — referential integrity is enforced at the application layer (Mongoose validators).
4. Merge tightly related tables into single collections to avoid expensive lookups.
5. Keep loosely related or high-volume data as separate collections to avoid unbounded document growth.

> *The original 12 tables collapse into 6 MongoDB collections: users, skills, projects, tasks, notifications, and skillmatchscores.*

6.1 Collection: users

Replaces: Users Table + UserSkills Junction Table + LearningRecommendations Table

The skills and learningRecommendations arrays are embedded directly into each user document, eliminating the need for the UserSkills and LearningRecommendations junction tables.

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Replaces UUID Primary Key
email	String, unique, required	Indexed for fast auth lookups
password_hash	String, required	Bcrypt hashed — no change
name	String, required	No change
role	String, enum: ['MEMBER','PROJECT_MANAGER','ADMIN']	Mongoose enum replaces PostgreSQL ENUM type
bio	String, optional	No change
profile_picture_url	String, optional	No change
availability_hours_per_week	Number, default: 40	Number replaces Integer
current_capacity_percentage	Number, min:0, max:100	No change
skills	Array of embedded SkillEntry objects	MERGED from UserSkills table — see sub-schema below

Field	Type / Constraint	Notes / Changes from PostgreSQL
learningRecommendations	Array of embedded Recommendation objects	MERGED from LearningRecommendations table — see sub-schema below
created_at	Date, default: Date.now	Timestamp replaces PostgreSQL TIMESTAMP
updated_at	Date, default: Date.now	Auto-updated via Mongoose middleware

Embedded: skills[] sub-schema (replaces UserSkills junction table)

Field	Type / Constraint	Notes / Changes from PostgreSQL
skill_id	ObjectId, ref: 'skills'	Reference to skills collection — no FK constraint
proficiency_level	String, enum: ['BEGINNER','INTERMEDIATE','ADVANCED','EXPERT']	Replaces PostgreSQL ENUM
verified	Boolean, default: false	No change
last_used	Date, optional	No change

Embedded: learningRecommendations[] sub-schema (replaces LearningRecommendations table)

Field	Type / Constraint	Notes / Changes from PostgreSQL
skill_id	ObjectId, ref: 'skills'	Reference — no FK
current_level	String, enum: ['NONE','BEGINNER','INTERMEDIATE','ADVANCED']	Mongoose enum
target_level	String, enum: ['BEGINNER','INTERMEDIATE','ADVANCED','EXPERT']	Mongoose enum
reason	String, optional	No change
course_url	String, optional	No change
course_name	String, optional	No change
priority	String, enum: ['LOW','MEDIUM','HIGH']	Mongoose enum
created_at	Date, default: Date.now	No change

> *REMOVED: Separate UserSkills table (6.3) and LearningRecommendations table (6.12) — both are now embedded in users.*

6.2 Collection: skills

Replaces: Skills Table only — no structural change, just adapted to MongoDB.

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Replaces UUID Primary Key
name	String, unique, required	Indexed — e.g., 'React', 'Python'
category	String, optional	e.g., 'Frontend', 'Backend', 'Soft Skills'
created_at	Date, default: Date.now	No change

> *This collection is the global skill taxonomy. Admins manage it. skills._id is referenced in users.skills[].skill_id, projects.requiredSkills[].skill_id, and users.learningRecommendations[].skill_id.*

6.3 Collection: projects

Replaces: Projects Table + ProjectSkills Junction Table + ProjectMembers Junction Table

Required skills and team members are embedded as arrays directly inside the project document, eliminating two junction tables.

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Replaces UUID Primary Key
name	String, required	No change
description	String, optional	No change
status	String, enum: ['PLANNING','ACTIVE','COMPLETED','ON_HOLD']	Mongoose enum replaces PostgreSQL ENUM
start_date	Date, optional	No change
end_date	Date, optional	No change
budget	Number, optional	Number replaces Decimal
created_by	ObjectId, ref: 'users'	Replaces UUID FK to Users table
ai_success_score	Number, min:0, max:100, default: null	No change
requiredSkills	Array of embedded RequiredSkill objects	MERGED from ProjectSkills table — see sub-schema
members	Array of embedded Member objects	MERGED from ProjectMembers table — see sub-schema
created_at	Date, default: Date.now	No change
updated_at	Date, default: Date.now	Auto-updated via Mongoose middleware

Embedded: requiredSkills[] sub-schema (replaces ProjectSkills junction table)

Field	Type / Constraint	Notes / Changes from PostgreSQL
skill_id	ObjectId, ref: 'skills'	Reference — no FK constraint
required_proficiency	String, enum: ['BEGINNER','INTERMEDIATE','ADVANCED','EXPERT']	Mongoose enum
is_critical	Boolean, default: false	No change

Embedded: members[] sub-schema (replaces ProjectMembers junction table)

Field	Type / Constraint	Notes / Changes from PostgreSQL
user_id	ObjectId, ref: 'users'	Reference — no FK constraint
role_in_project	String, optional	e.g., 'Frontend Developer', 'Team Lead'
assigned_at	Date, default: Date.now	Replaces PostgreSQL TIMESTAMP

> *REMOVED: Separate ProjectSkills table (6.5) and ProjectMembers table (6.6) — both embedded in projects.*

6.4 Collection: tasks

Replaces: Tasks Table + TaskDependencies Junction Table + Comments Table

Task dependencies are embedded as a simple ObjectId array. Comments are also embedded since they are always fetched with their task — embedding avoids an extra query on every task view.

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Replaces UUID Primary Key
project_id	ObjectId, ref: 'projects', required	Indexed for project-based queries
title	String, required	No change
description	String, optional	No change
assigned_to	ObjectId, ref: 'users', optional	Replaces UUID FK
status	String, enum: ['TODO','IN_PROGRESS','IN_REVIEW','DONE']	Mongoose enum
priority	String, enum: ['LOW','MEDIUM','HIGH','URGENT']	Mongoose enum
due_date	Date, optional	No change
estimated_hours	Number, optional	Number replaces Decimal
actual_hours	Number, default: 0	No change
parent_task_id	ObjectId, ref: 'tasks', optional	Self-reference for subtasks — no FK
dependencies	Array of ObjectId, ref: 'tasks'	MERGED from TaskDependencies table — simple array of task _id refs

Field	Type / Constraint	Notes / Changes from PostgreSQL
comments	Array of embedded Comment objects	MERGED from Comments table — see sub-schema
created_by	ObjectId, ref: 'users', required	Replaces UUID FK
created_at	Date, default: Date.now	No change
updated_at	Date, default: Date.now	Auto-updated via Mongoose middleware

Embedded: comments[] sub-schema (replaces Comments table)

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Each comment gets its own _id for direct reference
user_id	ObjectId, ref: 'users'	Replaces UUID FK
content	String, required	No change
created_at	Date, default: Date.now	No change
updated_at	Date, default: Date.now	No change

> *REMOVED: Separate TaskDependencies table (6.8) and Comments table (6.9) — dependencies is now an array of ObjectIds, comments are embedded in tasks. If comments become very large (100+), consider moving to a separate collection.*

6.5 Collection: notifications

Replaces: Notifications Table — kept as a separate collection since notifications are queried independently by user and can grow to large volumes.

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Replaces UUID Primary Key
user_id	ObjectId, ref: 'users', required	Indexed for per-user notification queries
type	String, enum: ['TASK_ASSIGNED','MENTION','COMMENT','DEADLINE','ALERT']	Mongoose enum
title	String, required	No change
message	String, required	No change
link	String, optional	URL to relevant resource — no change
is_read	Boolean, default: false	Indexed for unread counts
created_at	Date, default: Date.now	No change

> *Kept as a separate collection (not embedded in users) because a single user can accumulate thousands of notifications over time. Embedding would cause the user document to grow unboundedly.*

6.6 Collection: skillmatchscores

Replaces: SkillMatchScores Table — kept as a separate collection since it stores cached AI computation results per user-project pair.

Field	Type / Constraint	Notes / Changes from PostgreSQL
_id	ObjectId (auto-generated)	Replaces UUID Primary Key
user_id	ObjectId, ref: 'users', required	Compound index with project_id
project_id	ObjectId, ref: 'projects', required	Compound index with user_id
match_score	Number, min:0, max:100	No change
skill_coverage_percentage	Number, min:0, max:100	No change
availability_fit	Number, min:0, max:100	No change
workload_fit	Number, min:0, max:100	No change
calculated_at	Date, default: Date.now	Replaces PostgreSQL TIMESTAMP — TTL index recommended (auto-expire after 24h)

> *Add a TTL index on calculated_at (expireAfterSeconds: 86400) so stale match scores auto-expire and the AI recalculates fresh scores. This replaces manual cache invalidation logic.*

Summary: Original Tables vs MongoDB Collections

Original PostgreSQL Table	Action	MongoDB Destination
6.1 Users	**Kept**	users collection
6.2 Skills	**Kept**	skills collection
6.3 UserSkills	**MERGED**	Embedded in users.skills[]
6.4 Projects	**Kept**	projects collection
6.5 ProjectSkills	**MERGED**	Embedded in projects.requiredSkills[]
6.6 ProjectMembers	**MERGED**	Embedded in projects.members[]
6.7 Tasks	**Kept**	tasks collection
6.8 TaskDependencies	**MERGED**	Embedded in tasks.dependencies[]
6.9 Comments	**MERGED**	Embedded in tasks.comments[]
6.10 Notifications	**Kept**	notifications collection

Original PostgreSQL Table	Action	MongoDB Destination
6.11 SkillMatchScores	**Kept**	skillmatchscores collection
6.12 LearningRecommendations	**MERGED**	Embedded in users.learningRecommendations[]

Additional MongoDB-Specific Recommendations

1. **Indexes to create:**
   users: { email: 1 } unique index for auth lookups.
   tasks: { project_id: 1 } index for fetching all tasks of a project.
   notifications: { user_id: 1, is_read: 1 } compound index for notification bell queries.
   skillmatchscores: { user_id: 1, project_id: 1 } unique compound index + TTL on calculated_at.

2. **Security note:**
   The SRS mentioned SQL Injection Prevention via parameterized queries. MongoDB does not use SQL, but you must still sanitize inputs to prevent NoSQL injection. Use mongoose-sanitize or express-mongo-sanitize middleware on all incoming request bodies.

3. **Mongoose models:**
   Use Mongoose (ODM) to enforce schema validation at the application level since MongoDB itself is schema-less. Define enums, required fields, and min/max constraints in your Mongoose schemas to replicate the constraints the PostgreSQL schema provided natively.

4. **Deployment note:**
   The SRS specified PostgreSQL with read replicas. Replace this with MongoDB Atlas with a replica set (M10+ cluster) to achieve the same high availability and read scaling. Update the deployment section accordingly.